

$$\frac{\partial \mathcal{L}}{\partial \beta} = -\frac{2}{N} X^T \cdot (y - \hat{y})$$

```
coef_der = -(2/N) *  
np.dot(X_train.T,  
        (y_train - y_hat))
```

Demystifying Batch Gradient Descent

From Mathematical Formulation to Vectorised NumPy Implementation

The Optimisation Landscape

Batch Gradient Descent

- Computes the gradient using the entire dataset for a single update.
- Best for smaller datasets and convex functions.
- Focus of this module.

Stochastic Gradient Descent (SGD)

- Updates after every single row.
- Faster convergence, suited for massive datasets.

Mini-Batch Gradient Descent

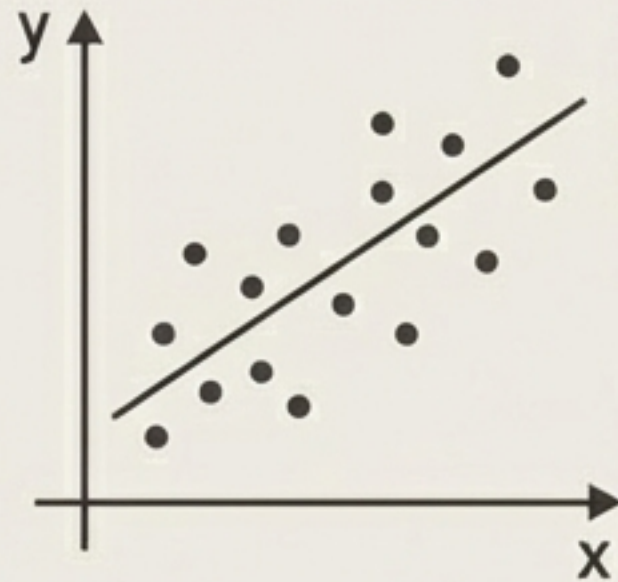
- The middle ground.
- Updates after a defined batch size (e.g., 30 rows).



Key Insight: We are focusing on **Batch Gradient Descent** to understand the foundational mathematics of n-dimensional optimisation before scaling to faster variants.

The N-Dimensional Challenge

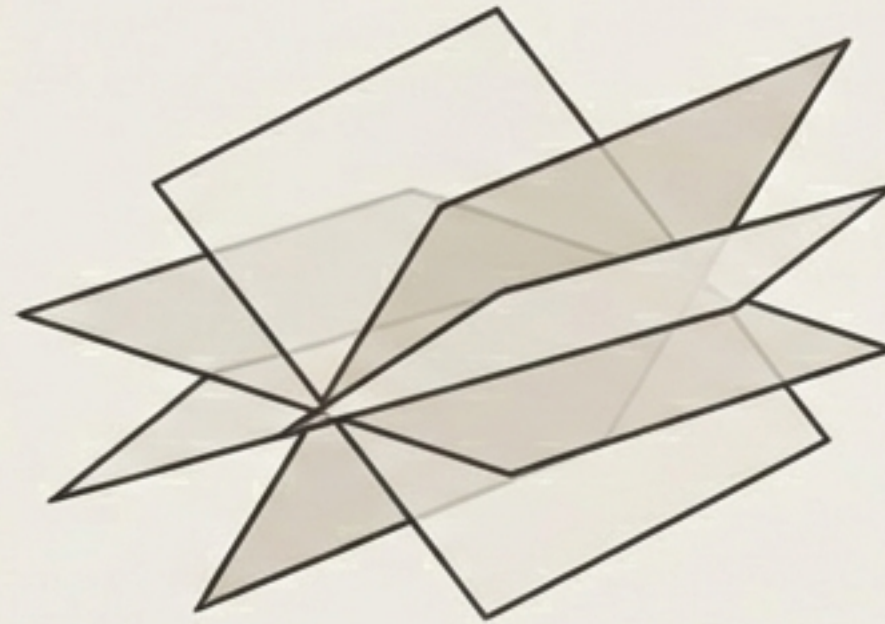
Simple 2D



1 Input Feature.
2 Coefficients to find (m, b).

$$y = mx + b$$

Multiple Linear Regression



CGPA	IQ	Gender	→ LPA/Salary
.	.	.	.
.	.	.	.
...	...	...	...

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Takeaway: For n features, we must simultaneously optimise n+1 parameters (β_0 to β_n).

The Core Update Rule

Step 1: Initialisation

Intercept (β_0) = 0
Coefficients ($\beta_1 \dots \beta_n$) = 1

Step 2: The Update Equation

$\text{new_value} = \text{old_value} - (\text{learning_rate} \times \text{slope})$

The Context

The 'slope' is the partial derivative of our Loss Function.

To move forward, we must define the loss and differentiate it for every single parameter.

Defining the Target: Mean Squared Error (MSE)

$$L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

The actual target value.

Total number of rows in the training data.

Our model's predicted value ($\beta_0 + \beta_1 x_1 \dots$).



Minimise through partial differentiation.

Deriving the Intercept (β_0)

Step 1: The Initial Setup

$$\frac{\partial L}{\partial \beta_0} = \frac{\partial}{\partial \beta_0} \left[\frac{1}{n} \sum (y_i - (\beta_0 + \beta_1 x_{i1} \dots))^2 \right]$$



Step 2: Chain Rule Applied

$$(-2/n) \sum (y_i - \hat{y}_i) \cdot (1)$$

Key Takeaway: The slope for the intercept is simply the mean of the errors, scaled by -2. It does not multiply against any specific input feature.

Deriving the Coefficients ($\beta_1 \dots \beta_n$)

$$\frac{\partial L}{\partial \beta_1} = (-2/n) \sum (y_i - \hat{y}_i) \cdot x_{i1}$$



The Column Multiplier

Unlike the intercept, deriving for β_1 leaves behind x_{i1} due to the chain rule.

Generalised Rule: To find the derivative for any coefficient β_k , multiply the raw error by the corresponding column k 's data values.

The Vectorisation Breakthrough

```
for i in range(rows):  
    for j in range(cols):  
        sum += X[i][j] * error
```

$O(N \times M)$ complexity.
Computationally expensive for
thousands of rows.

$$\mathbf{X} \cdot \boldsymbol{\beta}$$

Hardware-accelerated.
Executed instantly via NumPy.

Takeaway: We must discard iterative summation (Σ) and translate our calculus entirely into matrix operations.

Visualising the Matrix Math

$$\begin{array}{|c|} \hline X_{\text{train}} \\ \hline 353 \times 10 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline \beta \\ \hline 10 \times 1 \\ \hline \end{array} = \begin{array}{|c|} \hline \hat{y} \\ \hline 353 \times 1 \\ \hline \end{array}$$

Insight: Matrix dot products inherently compute the $\sum$ operation across all 10 dimensions simultaneously.

Coding from Scratch: Class Initialisation

Hyperparameters
passed by the
user upon
initialization.

```
class GDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs
```

Attributes
awaiting
population by
the fit() method.

Coding from Scratch: The Intercept Update

$$\beta_0 = \beta_0 - \alpha \left[\frac{-2}{n} \sum (y - \hat{y}) \right]$$


```
# Inside the fit() loop
y_hat = np.dot(X_train, self.coef_) + self.intercept_
intercept_der = -2 * np.mean(y_train - y_hat)
self.intercept_ = self.intercept - (self.lr * intercept_der)
```


Coding from Scratch: The Coefficient Update

$$\beta = \beta - \alpha \left[\frac{-2}{n} X^T \cdot (y - \hat{y}) \right]$$

```
coef_der = -(2/X_train.shape[0]) * np.dot(X_train.T, (y_train - y_hat))
self.coef_ = self.coef_ - (self.lr * coef_der)
```

The Transposition Trick

X must be transposed from (353, 10) to (10, 353) to properly multiply against the error vector (353, 1), yielding the required (10, 1) gradient vector.

Coding from Scratch: Making Predictions



Validation: Custom vs. Scikit-Learn

Scikit-Learn LinearRegression

Intercept: 152.13

Coefs: [0.038 -0.237 0.506 0.322
-0.741 0.465 0.111 0.203
0.746 0.076]

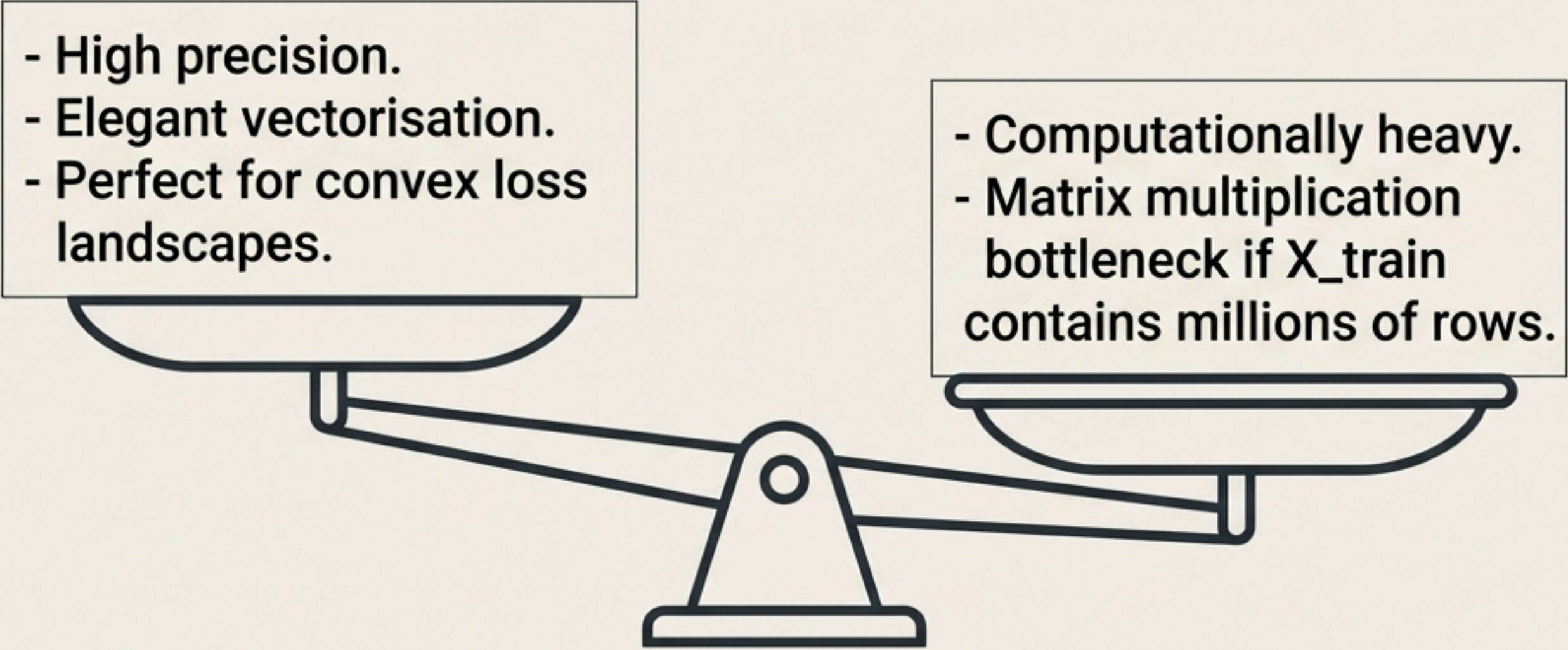
Custom GDRegressor | epochs=1000, lr=0.5

Intercept: 152.13

coef_: [0.038 -0.237 0.506 0.322
-0.741 -0.741 0.465 0.111 0.203
0.746 0.076]

VALIDATED

The Limits of Batch Gradient Descent

- 
- High precision.
 - Elegant vectorisation.
 - Perfect for convex loss landscapes.

- Computationally heavy.
- Matrix multiplication bottleneck if X_{train} contains millions of rows.

Next Steps: Batch Gradient Descent looks at the entire dataset for every step. To scale to millions of rows, we must look at **Stochastic Gradient Descent (SGD)**.